

Средства симуляции ЦП и ОС

Лекция №2



Державин Андрей
Шурыгин Антон

Квиз

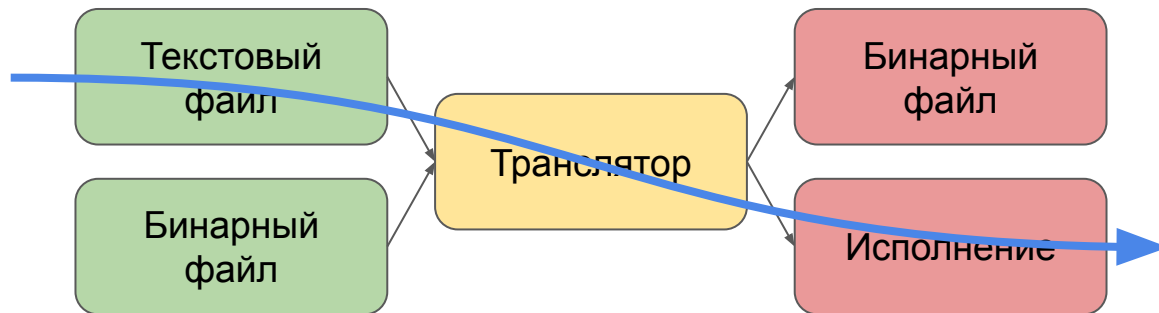
[Ссылка](#)

Определение

- Что такое интерпретатор?

Определение

- Что такое интерпретатор?

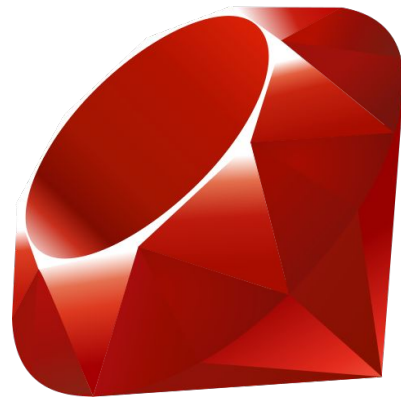
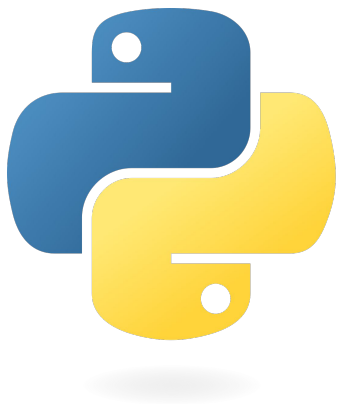


Определение

- Интерпретатор - один из видов трансляторов
- Какие существуют интерпретаторы?

Определение

- Интерпретатор - один из видов трансляторов
- Какие существуют интерпретаторы?



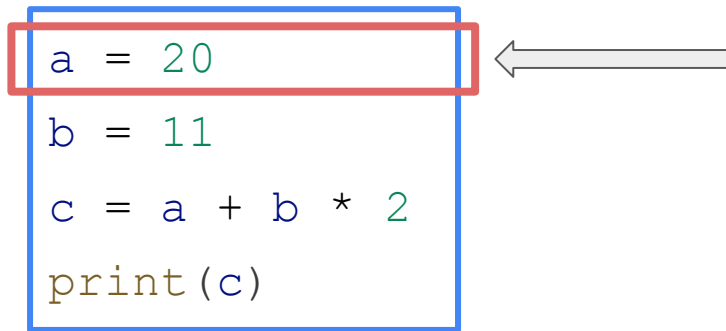
Определение

- Интерпретатор - один из видов трансляторов

```
a = 20  
b = 11  
c = a + b * 2  
print(c)
```

Определение

- Интерпретатор - один из видов трансляторов



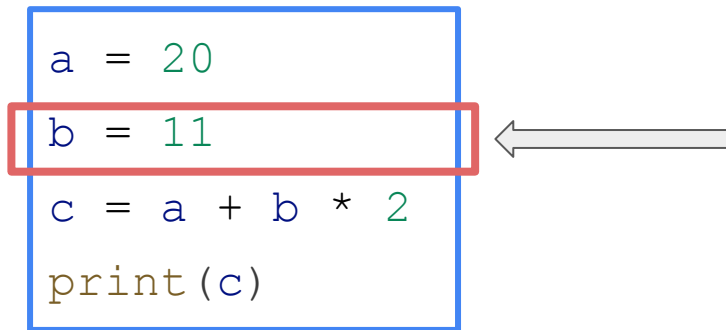
The diagram illustrates the execution of a code block. A blue rectangular box contains four lines of code: `a = 20`, `b = 11`, `c = a + b * 2`, and `print(c)`. The first line, `a = 20`, is highlighted with a red rectangular border. A gray arrow points from the right side of this red border towards the right edge of the slide, indicating the flow of execution or the current state of the interpreter.

```
a = 20
b = 11
c = a + b * 2
print(c)
```


Определение

- Интерпретатор - один из видов трансляторов

```
a = 20  
b = 11  
c = a + b * 2  
print(c)
```



Определение

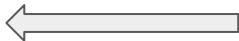
- Интерпретатор - один из видов трансляторов

```
a = 20
```

```
b = 11
```

```
c = a + b * 2
```

```
print(c)
```



Определение

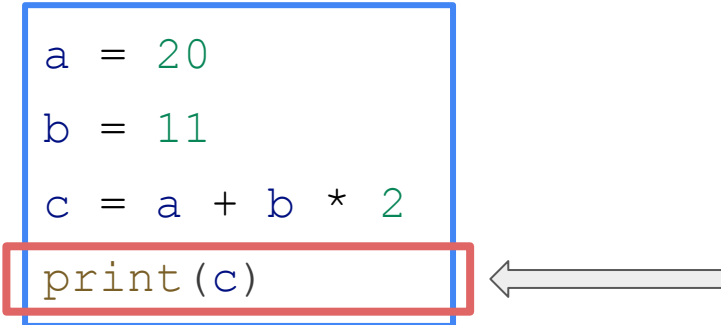
- Интерпретатор - один из видов трансляторов

```
a = 20
```

```
b = 11
```

```
c = a + b * 2
```

```
print(c)
```



Определение

- Интерпретатор - один из видов трансляторов

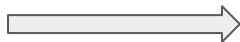
```
a = 20
b = 11
c = a + b * 2
print(c)
```

```
addi x1, x0, 20
addi x2, x0, 11
add x2, x2, x2
add x3, x2, x1
```

Определение

- Интерпретатор - один из видов трансляторов

```
a = 20  
b = 11  
c = a + b * 2  
print(c)
```



```
addi x1, x0, 20  
addi x2, x0, 11  
add x2, x2, x2  
add x3, x2, x1
```

Определение

- Интерпретатор - один из видов трансляторов

```
a = 20  
b = 11  
c = a + b * 2  
print(c)
```

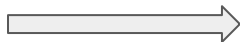


```
addi x1, x0, 20  
addi x2, x0, 11  
add x2, x2, x2  
add x3, x2, x1
```

Определение

- Интерпретатор - один из видов трансляторов

```
a = 20  
b = 11  
c = a + b * 2  
print(c)
```

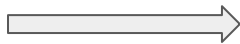


```
addi x1, x0, 20  
addi x2, x0, 11  
add x2, x2, x2  
add x3, x2, x1
```

Определение

- Интерпретатор - один из видов трансляторов

```
a = 20  
b = 11  
c = a + b * 2  
print(c)
```



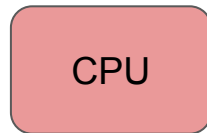
```
addi x1, x0, 20  
addi x2, x0, 11  
add x2, x2, x2  
add x3, x2, x1
```


Программная модель

- Какой главный модуль в нашей модели?

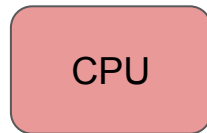
Программная модель

- Ядро исполняет инструкции



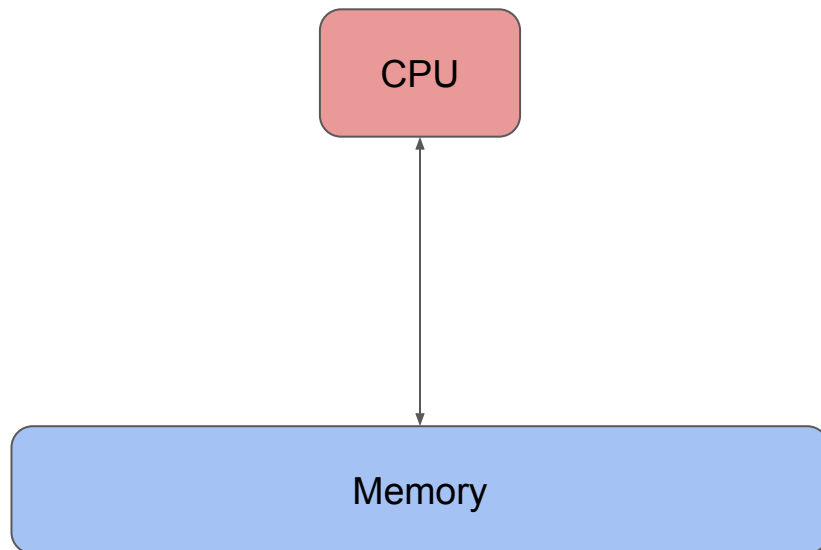
Программная модель

- Ядро исполняет инструкции
- Что ещё необходимо?



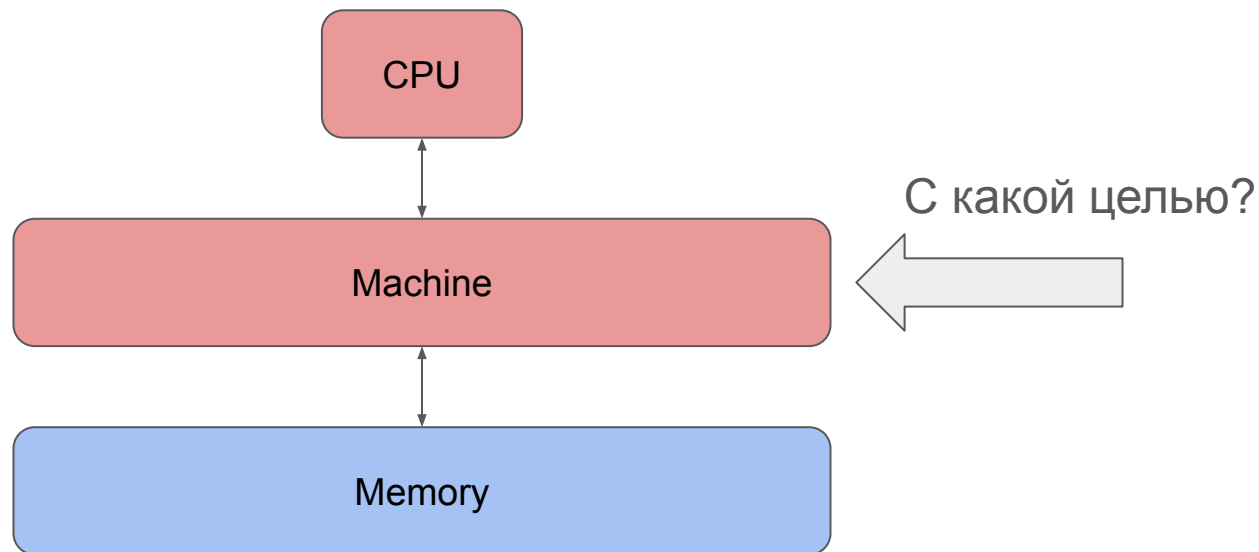
Программная модель

- Ядро исполняет инструкции
- Память - хранит код и данные



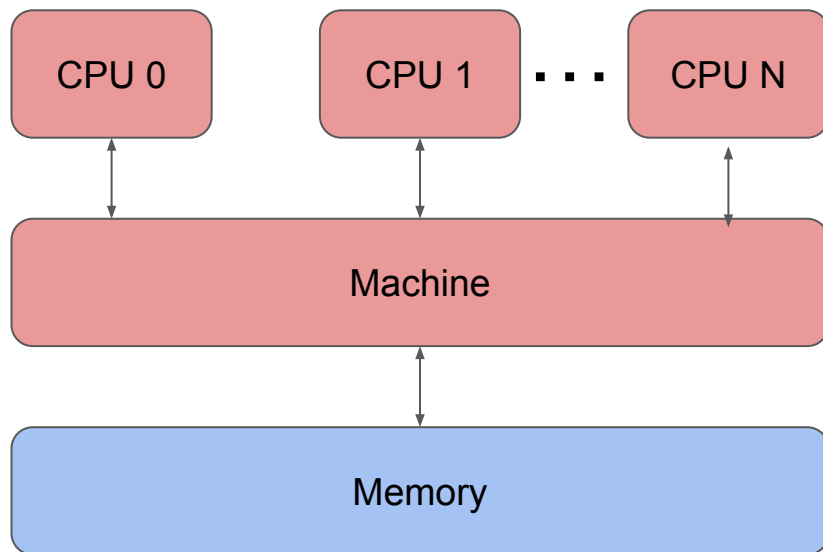
Программная модель

- Ядро исполняет инструкции
- Память - хранит код и данные



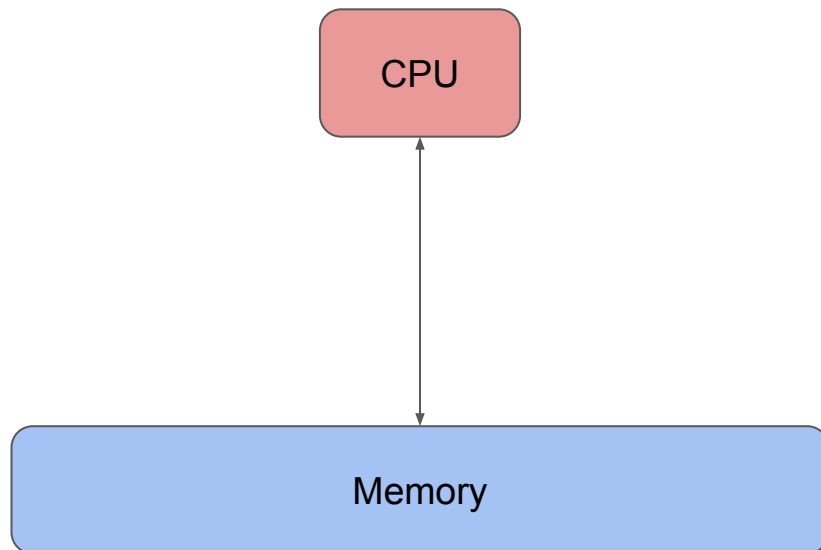
Программная модель

- Ядро исполняет инструкции
- Память - хранит код и данные



Программная модель

- Как бы вы реализовали простую модель?



Программная модель

- Простейшая модель на C

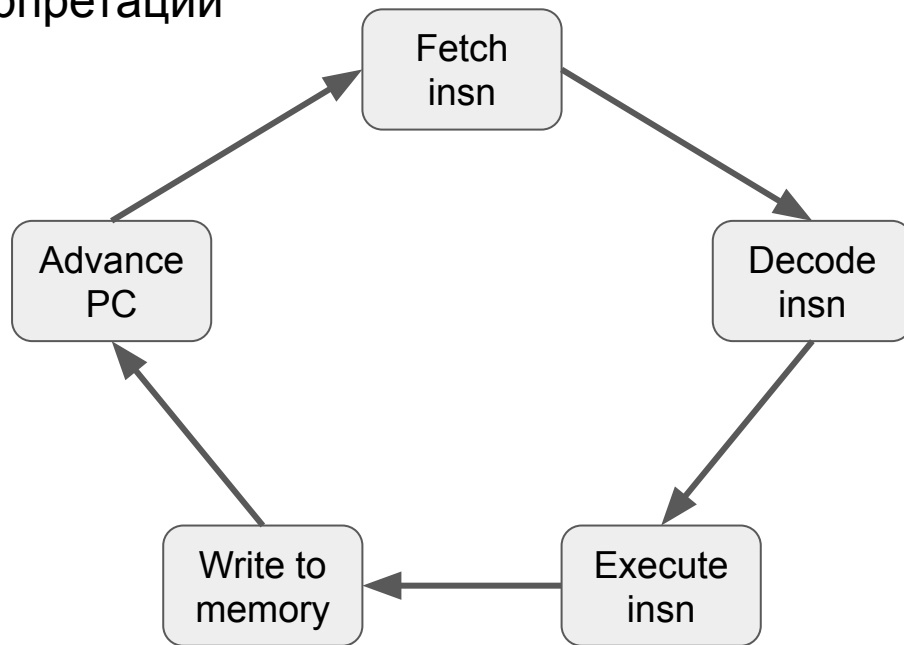
```
typedef uint32_t Register;
const size_t kNumRegs = 32;

struct CpuState {
    Register gpr_regs[kNumRegs];
    Memory *memory;
};

struct Memory {
    uint8_t *data;
};
```


Стадии интерпретатора

- PC хранит адрес текущей команды
- 5 стадий интерпретации



Стадии интерпретатора. Execute.

```
void execute(...) {  
    switch (opc) {  
        case ADD:  
            return do_add(...);  
        case SUB:  
            return do_sub(...);  
        case JUMP:  
            return do_jump(...);  
        // . . .  
    }  
}
```

Стадии интерпретатора. Write back

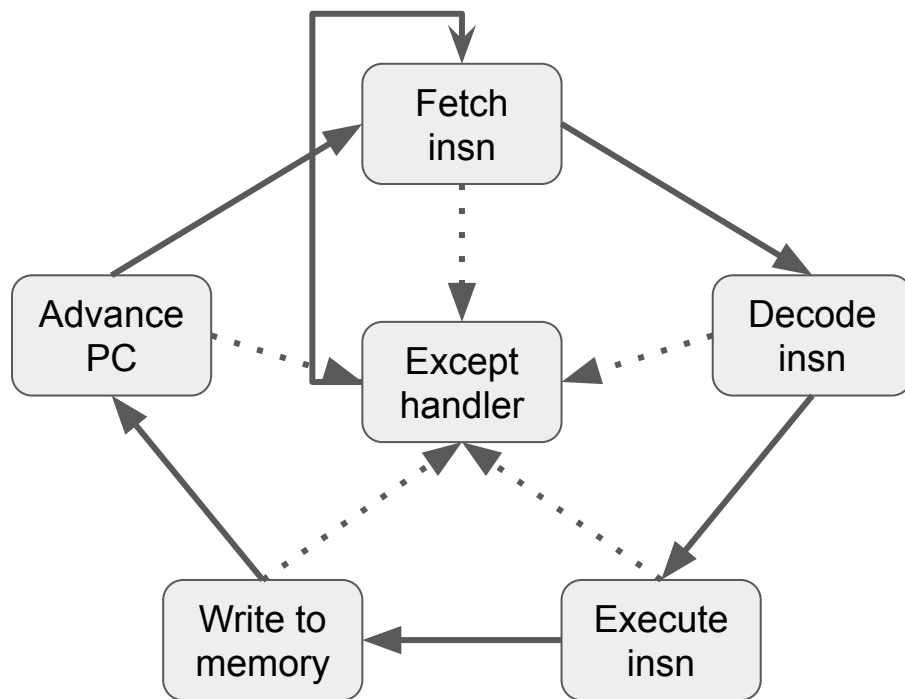
- Какие особенности может иметь доступ в память?

Стадии интерпретатора. Write back

- Режимы адресации ($\text{addr} = \text{base} + \text{offset} * \text{scale}$)
- Неявные операнды (push в x86)
- Различные требования по выравниванию (alignment)

Исключения

- Всегда ли программа выполняется последовательно
 - Исключения случаются...
- Разные виды:
 - Fault
 - Exception
 - Trap
 - Interrupt
 - Abort
- Могут требовать повторения инструкции
- Зависят от внешних событий

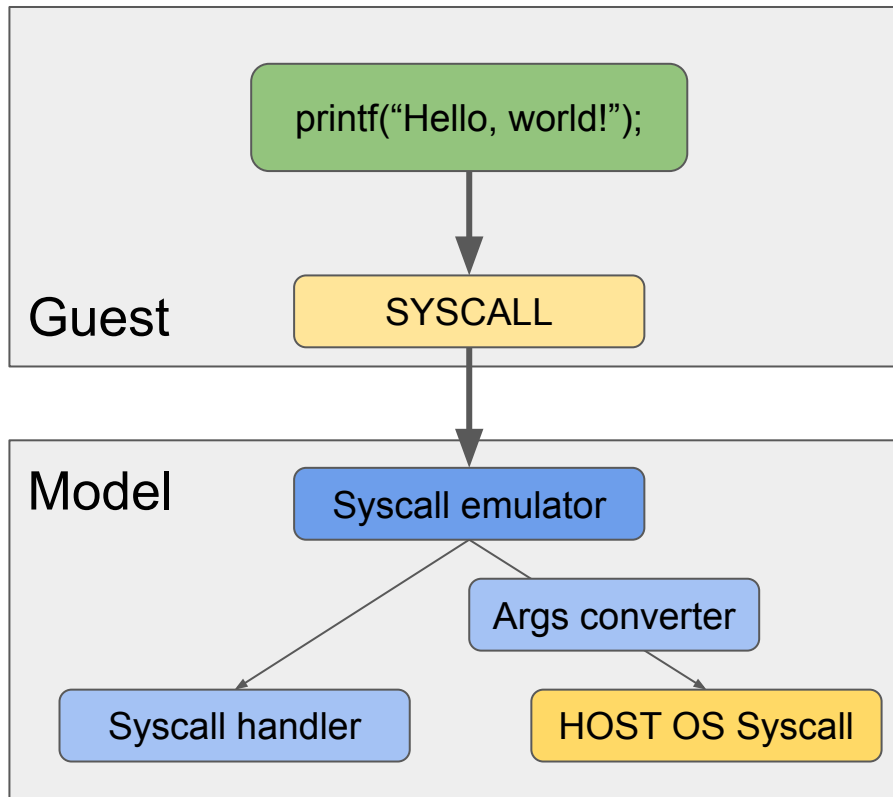


Исключения

- Будем вставлять везде проверку на исключения
- Как поддерживать обработку подобных ситуаций?
 - Требуется нелокальная передача управления
- `setjmp()`, `longjmp()` - good old C-style
- C++ style - exceptions?

Semihosting

- Почти все пользовательские приложения взаимодействуют с ОС
- Загрузка ОС не быстрый процесс
 - Требуется периферия
- Можно как то избежать моделирования ОС?
- Воспользуемся основным принципом моделирования
 - Абстракция



Ускорение интерпретатора

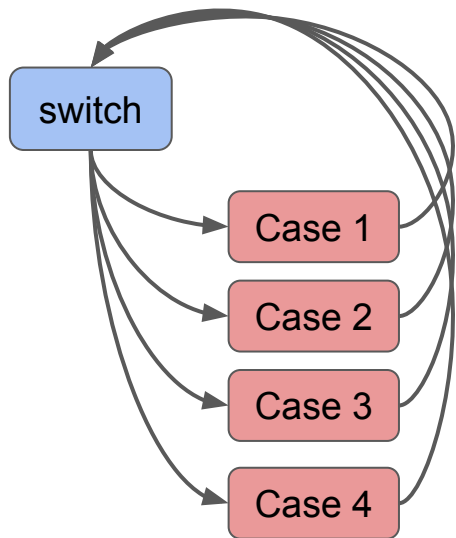
- Какие узкие места можно выделить в представленной модели?

Ускорение интерпретатора

- Шитый код (threaded code)
- Чем плох один большой switch?

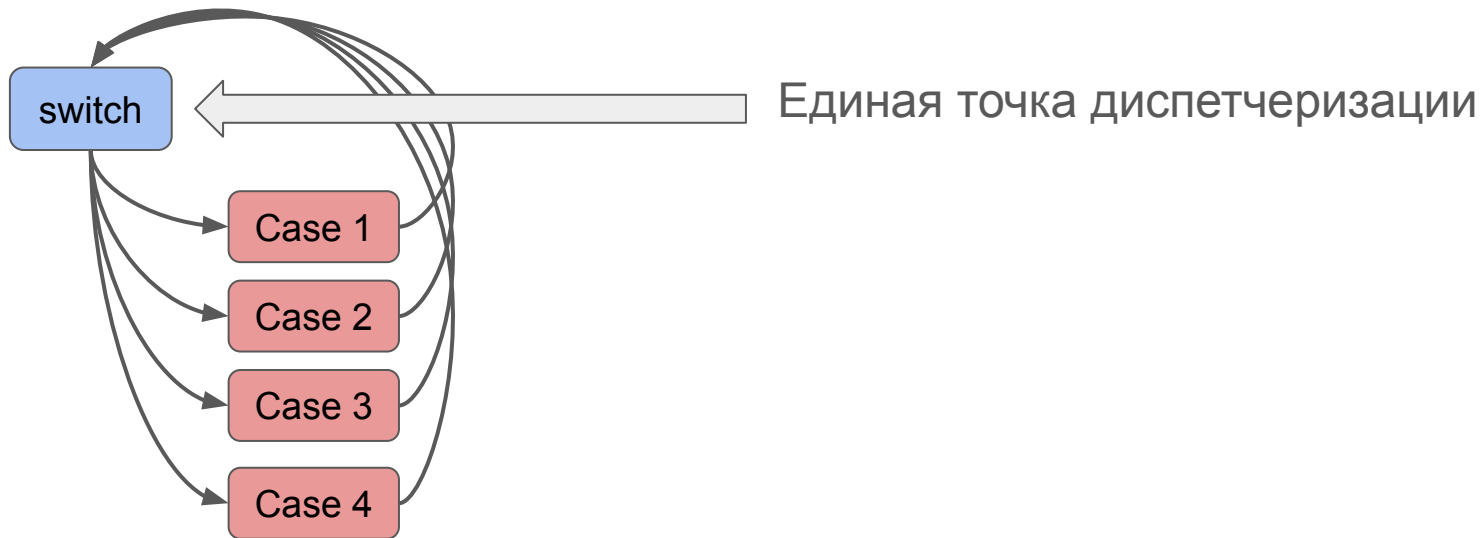
Ускорение интерпретатора

- Шитый код (threaded code)
- Чем плох один большой switch?



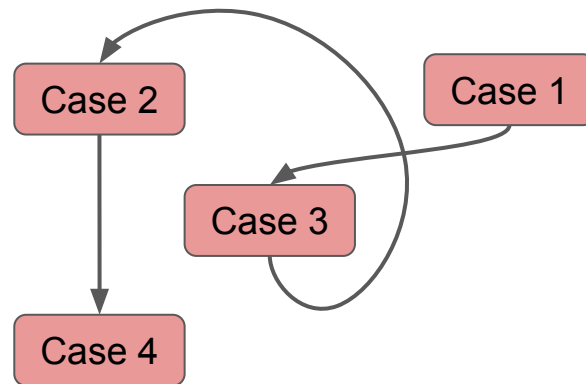
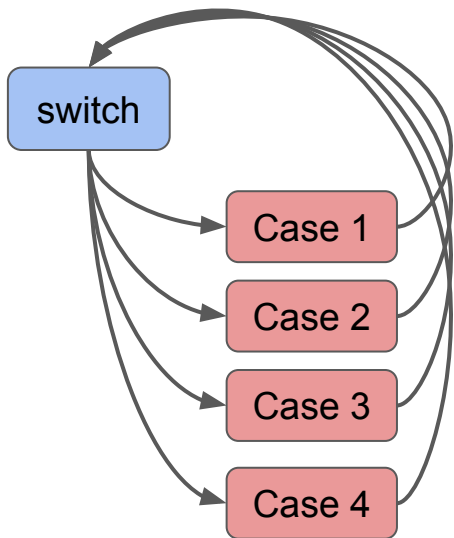
Ускорение интерпретатора

- Шитый код (threaded code)
- Чем плох один большой switch?



Ускорение интерпретатора

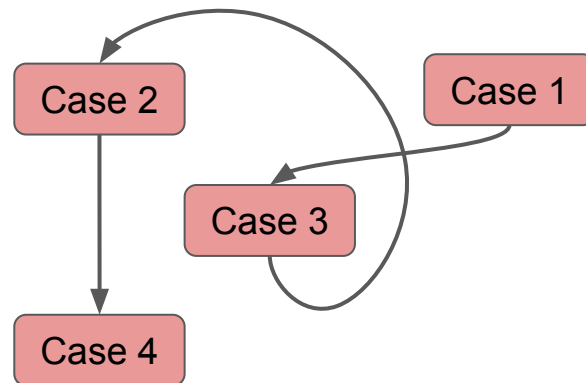
- Шитый код (threaded code)
- Введение нескольких точек диспетчеризации



Ускорение интерпретатора

- Шитый код (threaded code)
- Введение нескольких точек диспетчеризации
- Можно реализовать через GNU Extension
- Альтернатива - указатели на функции

```
DO_ADD:  
    ADD_handler(cpu, insn);  
    goto *label[++pc];
```



Ускорение интерпретатора

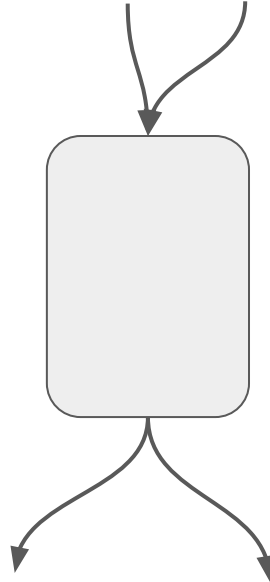
- Есть ли еще узкие места?
- Декодирование каждой исполняемой инструкции
- Идея: кэшировать результаты декодирования

Ускорение интерпретатора

- Что такое линейный участок кода?

Ускорение интерпретатора

- Линейный участок кода (Basic Block) - последовательность инструкций, имеющая **одну** точку входа и **одну** точку выхода



Ускорение интерпретатора

- *Линейный участок кода* (Basic Block) - последовательность инструкций, имеющая **одну** точку входа и **одну** точку выхода
- Какое свойство следуют из определения?

Ускорение интерпретатора

- *Линейный участок кода (Basic Block)* - последовательность инструкций, имеющая **одну** точку входа и **одну** точку выхода
- Инструкция из Базового блока выполняется $\Leftrightarrow$ выполняется первая инструкция Базового блока
- Как можно использовать это свойство для ускорения?

Ускорение интерпретатора

- *Линейный участок кода (Basic Block)* - последовательность инструкций, имеющая **одну** точку входа и **одну** точку выхода
- Инструкция из Базового блока выполняется $\Leftrightarrow$ выполняется первая инструкция Базового блока
- Идея: сохранять декодированные базовые блоки для повторного исполнения

Домашнее задание №1

- Безумный Архитектор
 - Реализовать симулятор ISA, которую спроектировал архитектор
 - В каждой СК набор из 14 различных инструкций
 - Кодировки между СК для одинаковых инструкций отличаются
 - В каждой СК есть несколько необычных RISC инструкций

Table of Contents

J	3
SYSCALL	4
STP	5
RORI	6
SLTI	7
ST	8
BDEP	9
CLS	10
ADD	11
BNE	12
BEQ	13
LD	14
AND	15
SSAT	16

Пример содержания спецификации

Домашнее задание №1



Домашнее задание №1

Возвращаясь к `struct CpuState`

- Регистровый файл
 - X – регистры общего назначения
 - Ширина регистра – 32 бита
 - Количество доступных регистров – 32
- PC – специальный 32-битный регистр, указывающий на исполняемую инструкцию

